Kubernetes

Kubernetes, often abbreviated as K8s, is an open-source system for automating deployment, scaling, and management of containerized applications.

Key Concepts

Pods

- The smallest deployable unit in Kubernetes.
- A group of one or more containers (such as Docker containers), with shared storage/network resources, and a specification for how to run the containers.
- Containers within a Pod share the same IP address and port space.

Nodes

- A Node is a worker machine in Kubernetes, which can be a virtual machine or a physical machine.
- Each Node is managed by the Control Plane and contains the necessary services to run Pods (Kubelet, Kube-proxy, and a container runtime like Docker).

Control Plane

- The components that manage the cluster's state, deployment, and scaling.
- Includes the API server, etcd (cluster state store), scheduler, and controller manager.

Deployments

- Provides declarative updates for Pods and ReplicaSets.
- You describe a desired state in a Deployment, and the Deployment Controller changes the actual state to the desired state.

Services

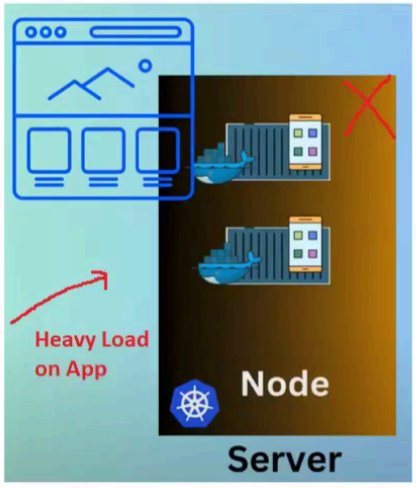
- An abstract way to expose an application running on a set of Pods as a network service.
- Services define a logical set of Pods and a policy by which to access them (sometimes called a micro-service).
- Common service types: `ClusterIP`, `NodePort`, `LoadBalancer`, and `ExternalName`.

Concept	Description	Analogy
Pod	Smallest deployable unit, group of containers	A house/apartment for containers
Node	Worker machine, runs Pods	A plot of land or server rack
Deployment	Manages desired state of Pods	A blueprint for a building
Service	Stable network endpoint for Pods	A street address for the application

What is Kubernetes

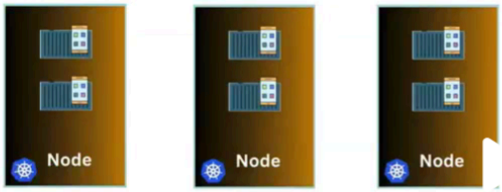
Kubernetes is a powerful open-source platform for automating the deployment, scaling, and operation of application containers.

Kubernetes also known as K8S which can manage containerized apps



If heavy traffic come on app, this single node will be failed

We need to now setup backup nodes immediately

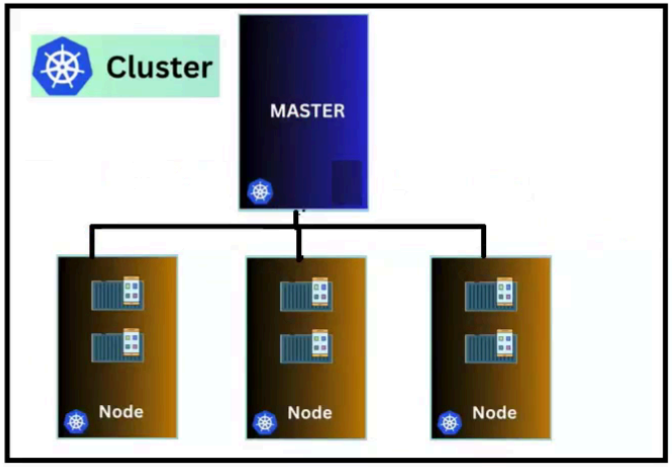


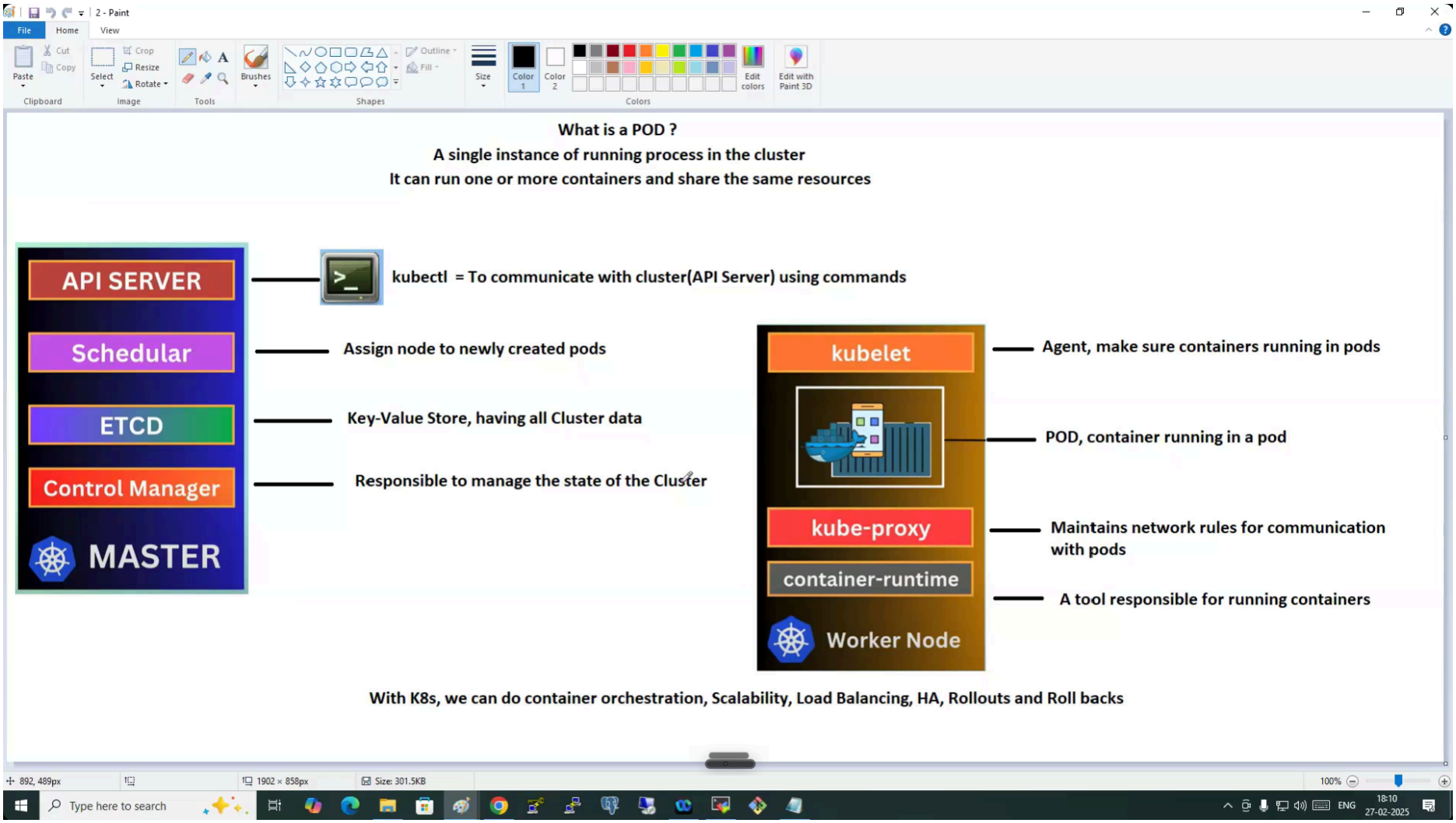
How to control and manage these nodes? Kubernetes

When we deploy kubernetes , we get cluster

Two important parts

1. Master (Control Panel)
2. Worker Nodes




2 - Paint
File
Home
View
Paste
Cut
Copy
Select
Crop
Resize
Rotate
Brushes
Outline
Fill
Size
Color 1
Color 2
Edit colors
Edit with Paint 3D
Clipboard
Image
Tools
Shapes
Colors
What is a POD ?
A single instance of running process in the cluster
It can run one or more containers and share the same resources
API SERVER
kubectl = To communicate with cluster(API Server) using commands
Schedular
Assign node to newly created pods
ETCD
Key-Value Store, having all Cluster data
Control Manager
Responsible to manage the state of the Cluster
MASTER
kubelet
Agent, make sure containers running in pods
POD, container running in a pod
kube-proxy
Maintains network rules for communication with pods
container-runtime
A tool responsible for running containers
Worker Node
With K8s, we can do container orchestration, Scalability, Load Balancing, HA, Rollouts and Roll backs
892, 489px
1902 × 858px
Size: 301.5KB
100%
Type here to search
ENG
18:10
27-02-2025

Kubernetes (K8s) — Structured Short Notes

1. What is Kubernetes?

Kubernetes (K8s) is an open-source container orchestration platform used to:

- Deploy containers
- Manage containers
- Scale applications
- Provide High Availability (HA)

Why Kubernetes is Needed?

Problem

If a single server/node gets:

- Heavy traffic
- Failure
- Crash

Application becomes unavailable.

Solution

Kubernetes creates:

- Multiple nodes
- Backup systems
- Load balancing
- Auto healing

This ensures applications remain available.

---

2. Kubernetes Cluster Architecture

A Kubernetes Cluster mainly contains:

Component	Purpose
Master Node	Controls the cluster
Worker Nodes	Runs applications/containers

---

3. Master Node Components

API Server

Main communication entry point of Kubernetes.

Purpose

- Receives commands
- Communicates with cluster

Example:

kubectl get pods

`kubectl` talks to API Server.

Scheduler

Assigns newly created Pods to Worker Nodes.

Responsibility

- Decides where Pods should run.

ETCD

Key-value database storing cluster information.

Stores

- Cluster state
- Configuration

- Secrets
- Node details

Controller Manager

Maintains desired cluster state.

Example

If a Pod crashes:

- Controller recreates it automatically.

---

4. Worker Node Components

Kubelet

Agent running on Worker Node.

Responsibility

- Ensures containers are running properly inside Pods.

Kube-Proxy

Handles networking rules.

Responsibility

- Communication between Pods
- Load balancing traffic

Container Runtime

Software responsible for running containers.

Examples

- Docker
- containerd
- CRI-O

---

5. What is a Pod?

A Pod is the smallest deployable unit in Kubernetes.

Definition

- A Pod can contain:
    - One container
    - Multiple containers
- Containers inside Pod share:
    - Network
    - Storage
    - Resources

Example

1 Pod = 1 or More Containers

---

6. Kubernetes Features

Kubernetes provides:

- Container orchestration
- Auto scaling
- Load balancing
- High Availability (HA)
- Self healing
- Rollouts
- Rollbacks

---

7. Basic Kubernetes Flow

Step-by-Step

Step 1

Developer sends command using:

kubectl

Step 2

API Server receives request.

Step 3

Scheduler selects Worker Node.

Step 4

Kubelet starts Pod on Worker Node.

Step 5

Kube-Proxy manages networking.

Pros of Kubernetes

- High scalability
- Self healing
- High availability
- Automatic load balancing
- Easy container orchestration
- Supports rolling updates

Cons of Kubernetes

- Complex for beginners
- Requires more resources
- Difficult troubleshooting initially
- Setup and networking can be complicated

The Interviewer’s Trap Question

Docker Swarm	Kubernetes
Easier setup	More powerful
Simple orchestration	Advanced orchestration
Beginner friendly	Enterprise-grade

The "Interrogation Shield"

1. Why Kubernetes instead of Docker alone?

Docker runs containers, but Kubernetes manages containers at large scale with automation, scaling, and self-healing.

2. What happens if a Pod crashes?

Kubernetes automatically recreates the Pod to maintain desired state.

3. What common mistake do beginners make?

Many beginners think Pods and containers are the same. Actually, Pods are wrappers that contain one or more containers.

Quick Memory Notes

Keyword	Meaning
Pod	Smallest deployable unit
API Server	Entry point

Scheduler	Assigns Pods
ETCD	Cluster database
Kubelet	Node agent
Kube-Proxy	Networking
Worker Node	Runs applications

The "Memory Anchor"

Kubernetes

“Manage Containers at Scale”

Pod

“Pod = Container House”

Think:

- Cluster = Company
- Master = Boss
- Worker Node = Employee
- Pod = Workspace
- Containers = Workers inside workspace

Kubernetes Networking, Volumes & Namespaces — Structured Notes

1. Kubernetes Networking Concepts

Kubernetes networking helps Pods and applications communicate inside and outside the cluster.

Cluster IP

Definition

Cluster IP provides an internal IP address for a Kubernetes Service.

Purpose

- Communication inside cluster only
- Internal application access

Important Point

Cannot be accessed directly from internet.

Use Case

- Backend API communication
- Internal microservices

Pros

- Secure internal communication
- Simple service discovery

Cons

- Not accessible externally

NodePort

Definition

NodePort exposes application using a fixed port on every node IP.

Purpose

Allows external users to access application.

Example

<Node-IP>:30000

Pros

- Easy external access
- Simple testing setup

Cons

- Limited port range
- Not ideal for production
- Manual port management

LoadBalancer

Definition

LoadBalancer creates external cloud load balancer for application access.

Purpose

Distributes traffic across Pods.

Common Cloud Support

- AWS
- Azure
- GCP

Pros

- Production-ready
- Automatic traffic distribution
- High availability

Cons

- Extra cloud cost
- Depends on cloud provider

The Interviewer's Trap Question

ClusterIP	NodePort	LoadBalancer
Internal access	External access via node port	External access via cloud LB
Default service type	Testing/demo usage	Production usage

---

2. Kubernetes Volumes

Definition

Volumes provide persistent storage for Pods.

Why Needed?

Without volumes:

- Data gets deleted when Pod restarts.

Purpose

- Store application data
- Maintain persistence
- Share data between containers

Storage Backends

Examples:

- Local storage
- AWS EBS
- Azure Disk
- NFS

Pros

- Data persistence
- Pod restart safety
- Supports cloud storage

Cons

- Storage management complexity
- Extra cloud cost

Real-World Example

- Database storage
- Log storage
- Upload files

---

3. Kubernetes Services

Definition

Services expose applications running inside Pods.

Purpose

Provides stable communication endpoint for Pods.

Types

Service Type	Access
ClusterIP	Internal
NodePort	External
LoadBalancer	External cloud access

Important Point

Pods can change IPs frequently, but Services provide stable access.

Pros

- Stable networking
- Load balancing support
- Easy Pod communication

Cons

- Networking troubleshooting can be difficult

---

4. Kubernetes Namespaces

Definition

Namespaces divide cluster resources between teams/users/projects.

Purpose

- Resource isolation
- Organization
- Multi-team management

Important Feature

Same resource names can exist in different namespaces.

Example

dev namespace → nginx
prod namespace → nginx

Both can exist simultaneously.

Pros

- Better organization
- Team isolation
- Easier management

Cons

- Namespace confusion for beginners
- Wrong namespace causes deployment issues

The "Interrogation Shield"

1. Why use Services in Kubernetes?

Pods are temporary and their IPs can change. Services provide stable access and load balancing.

2. Why are Volumes important?

Volumes prevent data loss when Pods restart or crash.

3. What common mistake do beginners make with Namespaces?

Beginners often deploy resources in wrong namespace and cannot find Pods or Services later.

Quick Memory Notes

Concept	Purpose
ClusterIP	Internal communication
NodePort	External access
LoadBalancer	Cloud external access
Volume	Persistent storage
Service	Stable Pod access
Namespace	Resource separation

The "Memory Anchor"

Networking

“Cluster Inside, Node Outside, LoadBalancer Worldwide”

Volume

“No Volume = No Data”

Namespace

“Namespaces = Separate Rooms”

Think:

- Cluster = Building
- Namespace = Different rooms for different teams

Kubernetes Cluster Creation — Structured Notes

1. Kubernetes Cluster Types

Kubernetes clusters can be created in two ways:

Type	Description
Self-Managed	We create and manage cluster manually
Cloud-Based	Cloud provider manages cluster

---

2. Self-Managed Kubernetes

Definition

In self-managed setup, we are responsible for:

- Installation
- Configuration
- Upgrades
- Maintenance
- Troubleshooting

Tools for Self-Managed Clusters

Minikube

Single-node Kubernetes cluster.

Purpose

- Learning Kubernetes
- Local testing
- Beginner practice

---

kubeadm

Creates multi-node Kubernetes cluster manually.

Purpose

- Production-like setup
- Manual cluster management

---

kops

Creates multi-node Kubernetes cluster using automation.

Purpose

- Automated Kubernetes deployment
- Mostly used with AWS

Pros of Self-Managed Clusters

- Full control
- Better customization
- Learning internal Kubernetes architecture

Cons of Self-Managed Clusters

- Complex setup
- Maintenance responsibility
- Upgrade difficulty
- More troubleshooting effort

---

3. Cloud-Based Kubernetes

Definition

Cloud providers manage Kubernetes infrastructure.

Popular Cloud Kubernetes Services

Cloud Provider	Kubernetes Service
AWS	EKS (Elastic Kubernetes Service)
Azure	AKS (Azure Kubernetes Service)
Google Cloud	GKS (Google Kubernetes Service)

Responsibility of Cloud Provider

Cloud provider manages:

- Control plane
- High availability
- Upgrades
- Monitoring basics

Pros of Cloud-Based Clusters

- Easier management

- Faster setup
- Better scalability
- Automatic upgrades
- High availability support

Cons of Cloud-Based Clusters

- Higher cloud cost
- Less low-level control
- Vendor dependency

---

4. kubectl

Definition

`kubectl` is the command-line tool used to communicate with Kubernetes cluster.

Purpose

Used to:

- Create resources
- Manage Pods
- Check cluster status
- Deploy applications

Example Commands

Get Pods

kubectl get pods

Get Nodes

kubectl get nodes

Deploy Application

kubectl apply -f deployment.yml

Pros of kubectl

- Powerful cluster management
- Full Kubernetes control
- Easy automation scripting

Cons of kubectl

- Large command set
- Hard for beginners initially

The Interviewer's Trap Question

Minikube	kubeadm	kops
Single node	Manual multi-node	Automated multi-node
Learning	Production practice	Cloud automation

Cloud vs Self-Managed

Self-Managed	Cloud-Based
Full control	Easy management

Manual maintenance	Provider managed

The "Interrogation Shield"

1. When should you use Minikube?

Use Minikube for local learning, testing, and Kubernetes practice on a single machine.

2. Why do companies prefer EKS/AKS/GKS?

Managed Kubernetes reduces operational overhead because cloud providers handle upgrades, HA, and control plane management.

3. What common mistake do beginners make?

Many beginners think Minikube is production-ready. Actually, it is mainly for local learning and testing.

Quick Memory Notes

Tool	Purpose
Minikube	Single-node practice
kubeadm	Manual multi-node setup
kops	Automated cluster setup
EKS	AWS Kubernetes
AKS	Azure Kubernetes
GKS	Google Kubernetes
kubectl	Kubernetes CLI tool

The "Memory Anchor"

Cluster Types

“Self Manage or Cloud Manage”

---

kubectl

“kubectl = Kubernetes Remote Control”

Think:

- Cluster = Car
- kubectl = Steering wheel
- Kubernetes = Driver system